



Università degli Studi di Napoli Parthenope

Corso di Laurea in Ingegneria e Scienze Informatiche per la Cybersecurity

AutoGarage

Sistema informativo per la gestione di
una concessionaria automobilistica

Esame di Basi di Dati - Prof. Daniele Granata

Analisi dei requisiti (1/2)

AutoGarage è un sistema informativo sviluppato per digitalizzare e ottimizzare la gestione delle attività di una concessionaria automobilistica multi-sede. Principali entità del sistema:

- **Utente:** rappresenta le persone che interagiscono con il sistema. Gli utenti possono assumere il ruolo di Cliente, Venditore o Meccanico.
- **Auto:** contiene tutte le informazioni relative ai veicoli presenti nel catalogo della concessionaria, come marca, modello, anno di immatricolazione, alimentazione, prezzo, stato e sede di appartenenza.
- **Sede:** identifica le diverse filiali della concessionaria e consente di distribuire il parco auto sul territorio.
- **Test Drive:** gestisce le prenotazioni effettuate dai clienti per provare un determinato veicolo.

Analisi dei requisiti (2/2)

- **Vendita:** registra le operazioni di acquisto dei veicoli effettuate dai clienti.
- **Noleggio:** consente la gestione dei contratti di noleggio associati alle automobili disponibili.
- **Intervento:** tiene traccia delle attività di manutenzione e riparazione effettuate dai meccanici sui veicoli.
- **Recensione:** permette ai clienti di esprimere una valutazione sui veicoli presenti nel catalogo.
- **Fornitore e Fornitura:** permettono di monitorare la provenienza dei veicoli e le operazioni di approvvigionamento.

Le entità individuate costituiscono la base per la progettazione del modello E-R.

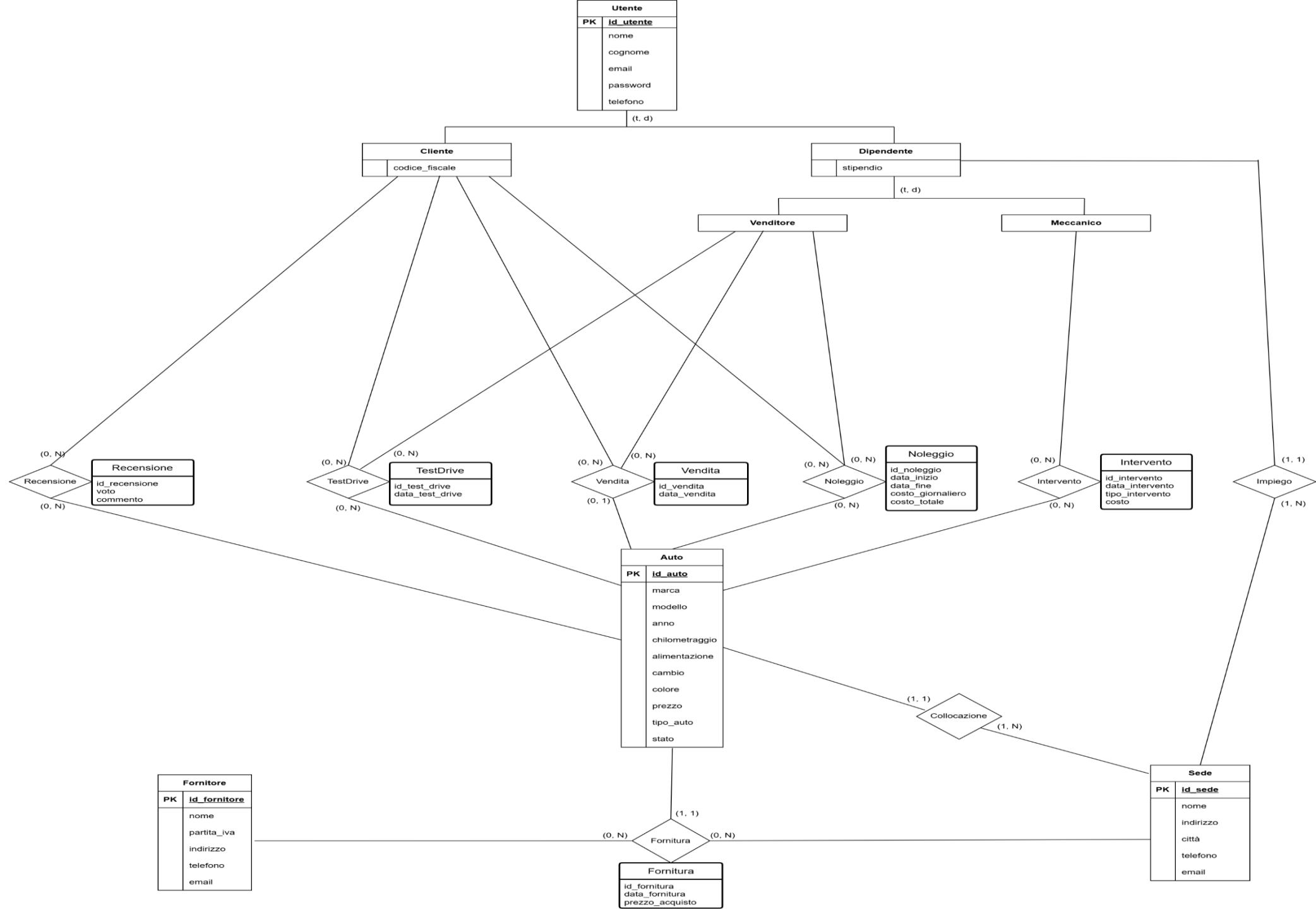
Modello E-R Base

Il modello E-R base rappresenta una prima formalizzazione dei requisiti emersi dall'analisi del dominio applicativo.

In questa fase sono state individuate le principali entità del sistema e le relative relazioni.

Il modello presenta una doppia generalizzazione/specializzazione:

- **Utente → Cliente, Dipendente**
- **Dipendente → Venditore, Meccanico**



Risoluzione della generalizzazione Dipendente – Venditore/Meccanico (1/3)

Scenario 1 – Accorpamento al padre

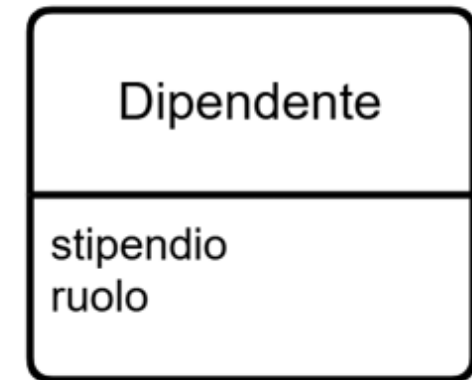
Le entità **Venditore** e **Meccanico** vengono eliminate e sostituite da un attributo **ruolo** all'interno dell'entità **Dipendente**.

Vantaggi:

- Riduzione del numero di entità e del numero di tabelle nel modello logico
- Nessuna duplicazione degli attributi comuni
- Query più semplici
- Maggiore flessibilità nell'introduzione di nuovi ruoli

Svantaggi:

- Vincoli gestiti tramite l'attributo ruolo
- Minore separazione concettuale dei ruoli



Risoluzione della generalizzazione Dipendente – Venditore/Meccanico (2/3)

Scenario 2 – Accorpamento alle figlie

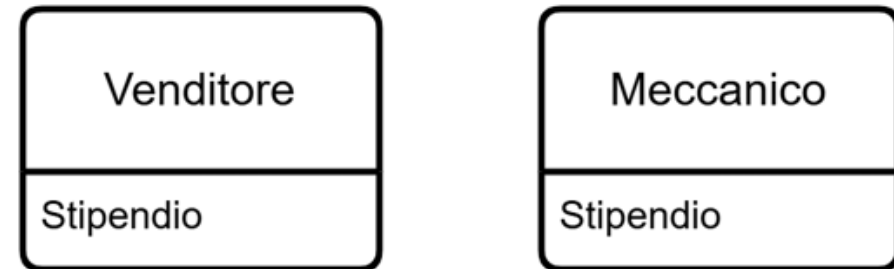
L'entità **Dipendente** viene eliminata e gli attributi comuni vengono trasferiti alle entità **Venditore** e **Meccanico**.

Vantaggi:

- Separazione esplicita dei ruoli
- Vincoli specifici applicabili direttamente alle entità

Svantaggi:

- Duplicazione degli attributi comuni
- Maggiore complessità gestionale
- Minore flessibilità nell'introduzione di nuovi ruoli

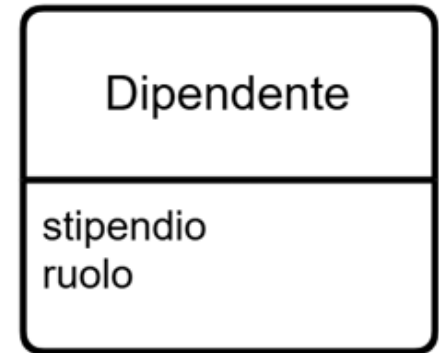


Risoluzione della generalizzazione Dipendente – Venditore/Meccanico (3/3)

Scelta adottata: accorpamento al padre

Motivazioni:

- Venditori e meccanici non possiedono attributi specifici rilevanti
- La distinzione riguarda principalmente le operazioni svolte
- Riduzione della complessità del modello
- Maggiore semplicità di implementazione



I vincoli semantici vengono mantenuti imponendo che solo i venditori possano gestire vendite, noleggi e test drive, mentre solo i meccanici possono effettuare interventi.

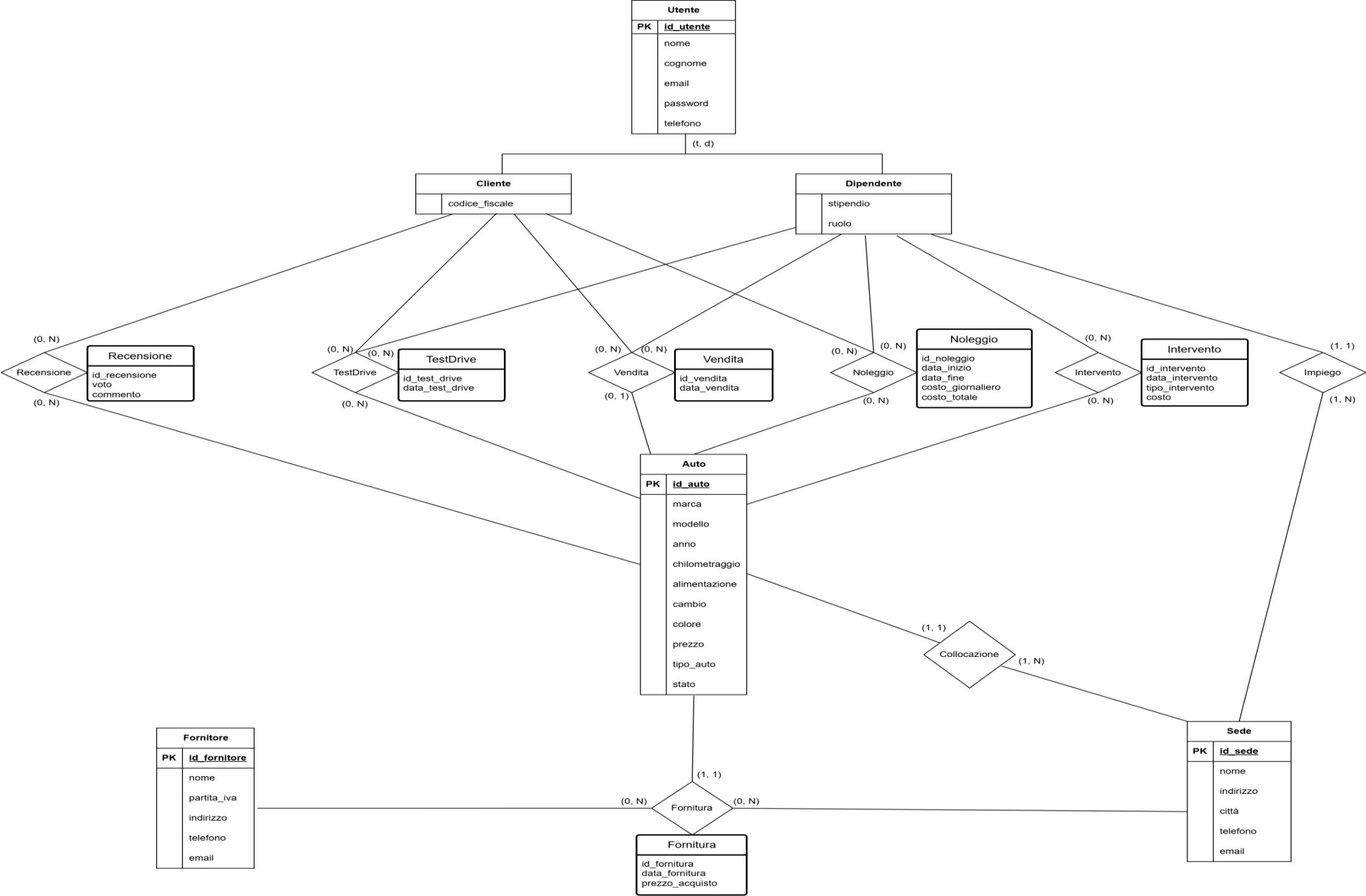
Modello E-R Intermedio

Il modello E-R intermedio deriva dalla ristrutturazione del modello base attraverso la risoluzione della generalizzazione **Dipendente – Venditore/Meccanico**.

In particolare, le entità Venditore e Meccanico sono state accorpate all'entità Dipendente mediante l'introduzione dell'attributo **ruolo**, riducendo la complessità del modello e semplificando la gestione del personale.

Il modello presenta un'ultima generalizzazione/specializzazione da risolvere:

- **Utente → Cliente, Dipendente**



Risoluzione della generalizzazione Utente – Cliente/Dipendente (1/3)

Scenario 1 – Accorpamento al padre

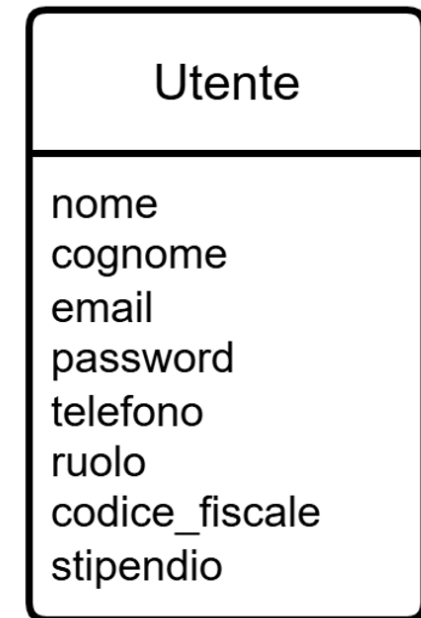
Le entità **Cliente** e **Dipendente** vengono eliminate e accorpate all'interno dell'entità **Utente**. La distinzione tra le diverse tipologie di utenti viene gestita tramite l'attributo **ruolo**, che può assumere i valori cliente, venditore e meccanico.

Vantaggi:

- Riduzione del numero di entità e del numero di tabelle nel modello logico
- Gestione centralizzata di tutti gli utenti
- Eliminazione delle join tra Utente, Cliente e Dipendente
- Maggiore semplicità nelle operazioni di autenticazione
- Maggiore flessibilità nell'introduzione di nuovi ruoli

Svantaggi:

- Presenza di attributi non sempre valorizzati
- Necessità di vincoli basati sul ruolo
- Minore separazione concettuale tra clienti e dipendenti



Risoluzione della generalizzazione Utente – Cliente/Dipendente (2/3)

Scenario 2 – Accorpamento alle figlie

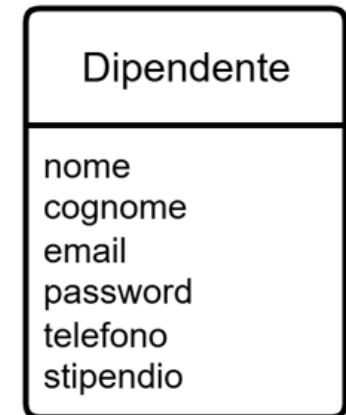
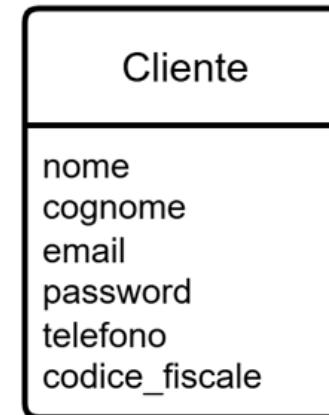
L'entità **Utente** viene eliminata e gli attributi comuni vengono trasferiti alle entità **Cliente** e **Dipendente**. In questo modo le due categorie vengono rappresentate come entità autonome e indipendenti.

Vantaggi:

- Separazione esplicita tra clienti e dipendenti
- Possibilità di applicare vincoli specifici alle singole entità
- Maggiore chiarezza concettuale del modello
- Maggiore specializzazione delle categorie di utenti

Svantaggi:

- Duplicazione degli attributi comuni
- Maggiore complessità generale
- Necessità di interrogare più entità per ottenere l'elenco completo degli utenti
- Minore flessibilità nell'introduzione di nuovi ruoli



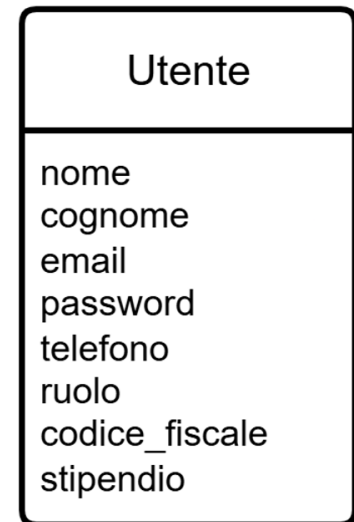
Risoluzione della generalizzazione Utente – Cliente/Dipendente (3/3)

Scelta adottata: accorpamento al padre

Motivazioni:

- Cliente e Dipendente condividono la maggior parte degli attributi comuni
- Riduzione del numero di entità e della complessità del modello
- Eliminazione della duplicazione dei dati
- Maggiore semplicità nelle operazioni di autenticazione e gestione utenti
- Maggiore flessibilità nell'introduzione di nuovi ruoli

I vincoli semantici vengono mantenuti attraverso l'attributo **ruolo**, che distingue le diverse tipologie di utenti e consente di limitare l'accesso alle funzionalità in base ai permessi associati.

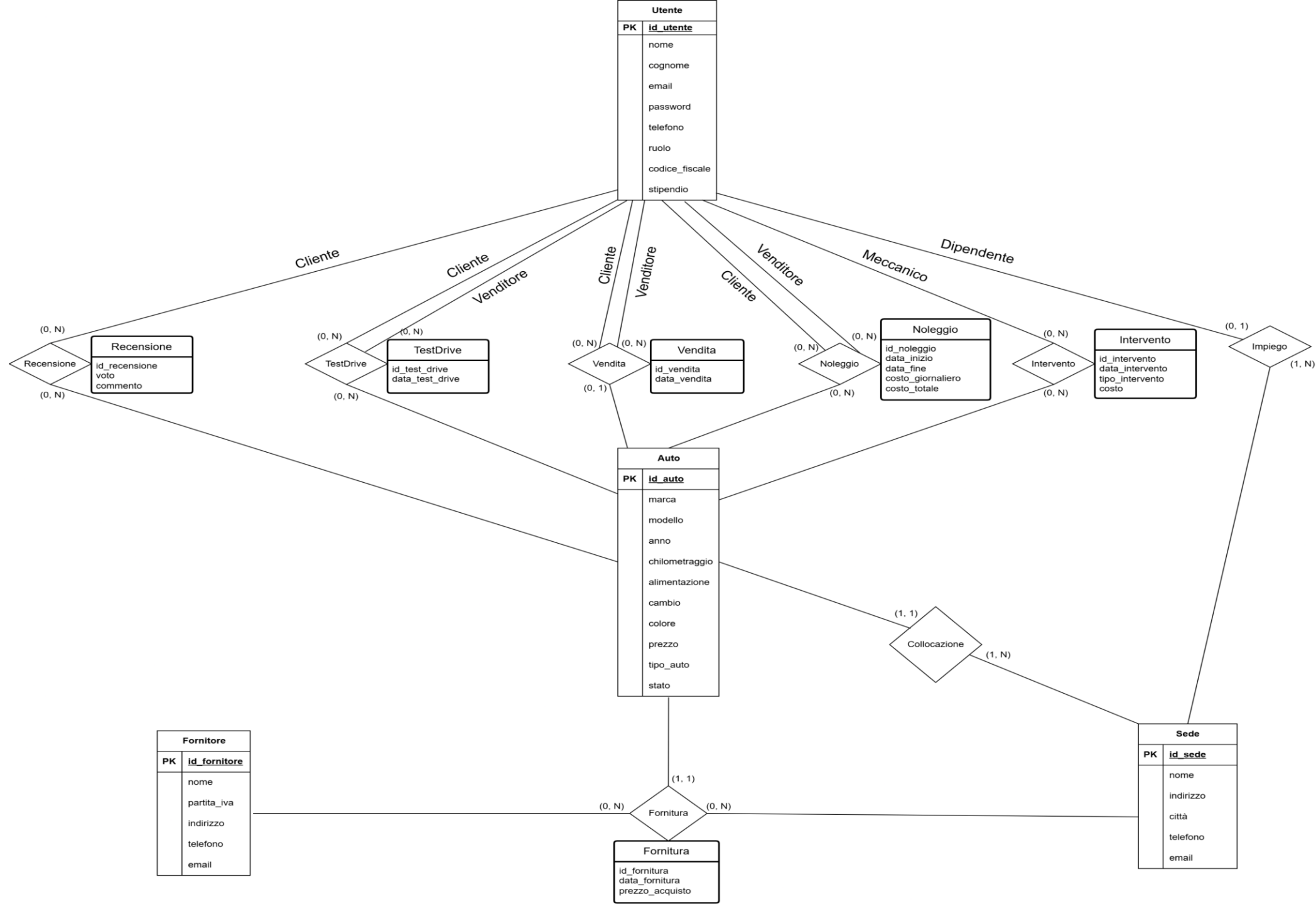


Modello E-R Finale

Il modello E-R finale deriva dalla completa ristrutturazione del modello intermedio attraverso la risoluzione della generalizzazione **Utente** → **Cliente/Dipendente**.

Le entità **Cliente** e **Dipendente** sono state accorpate nell'entità **Utente**, introducendo l'attributo **ruolo** per distinguere clienti, venditori e meccanici.

Nel modello finale tutte le relazioni fanno quindi riferimento all'entità **Utente**, mantenendo i vincoli semantici attraverso il ruolo associato a ciascun utente.



Sintesi del modello E-R Finale

Caratteristiche principali:

- Unica entità **Utente** con attributo **ruolo**
- Gestione centralizzata di clienti, venditori e meccanici
- Relazioni principali con **Auto**, **Sede**, **Vendita**, **Noleggio**, **Test Drive**, **Intervento** e **Recensione**
- Riduzione della complessità rispetto al modello base
- Vincoli semantici mantenuti tramite ruolo e cardinalità

Modello logico relazionale

- **Utente** (id_utente, nome, cognome, email, password, telefono, ruolo, codice_fiscale, stipendio)
- **Auto** (id_auto, marca, modello, anno, chilometraggio, alimentazione, cambio, colore, prezzo, tipo_auto, stato, id_sede: Sede)
- **Fornitore** (id_fornitore, nome, partita_iva, indirizzo, telefono, email)
- **Sede** (id_sede, nome, indirizzo, città, telefono, email)
- **Recensione** (id_recensione, voto, commento, id_cliente: Utente, id_auto: Auto)
- **TestDrive** (id_test_drive, data_test_drive, id_cliente: Utente, id_auto: Auto, id_venditore: Utente)
- **Vendita** (id_vendita, data_vendita, id_cliente: Utente, id_auto: Auto, id_venditore: Utente)
- **Noleggio** (id_noleggio, data_inizio, data_fine, costo_giornaliero, costo_totale, id_cliente: Utente, id_auto: Auto, id_venditore: Utente)
- **Intervento** (id_intervento, data_intervento, tipo_intervento, costo, id_auto: Auto, id_meccanico: Utente)
- **Fornitura** (id_fornitura, data_fornitura, prezzo_acquisto, id_fornitore: Fornitore, id_auto: Auto, id_sede: Sede)
- **Impiego** (id_dipendente: Utente, id_sede: Sede)

Vincoli interrelazionali (1/2)

Gestione dei ruoli:

- Gli utenti con ruolo **cliente** possono partecipare a recensioni, test drive, vendite e noleggi.
- Gli utenti con ruolo **venditore** possono gestire vendite, noleggi e test drive.
- Gli utenti con ruolo **meccanico** possono eseguire interventi.
- Ogni dipendente deve essere assegnato a una sola sede.

Gestione delle forniture:

- Ogni auto deve provenire da una sola fornitura.
- Ogni fornitura è associata a un solo fornitore, una sola auto e una sola sede.

Vincoli interrelazionali (2/2)

Gestione delle automobili:

- Un'auto può essere venduta una sola volta.
- Un'auto venduta non può essere noleggiata o prenotata per un test drive.
- Un'auto in manutenzione non può essere noleggiata o prenotata per un test drive.

Gestione delle operazioni:

- Due noleggi della stessa auto non possono avere periodi sovrapposti.
- Durante un noleggio l'auto non può essere venduta.
- Un cliente può recensire un'auto solo se l'ha acquistata, noleggiata o provata tramite test drive.

Trigger implementati

I trigger sono stati utilizzati per implementare alcuni vincoli semantici non esprimibili tramite semplici chiavi esterne.

Controllo dei ruoli:

- **before_insert_vendita**: verifica che la vendita coinvolga un cliente e un venditore.
- **before_insert_testdrive**: verifica che il test drive sia richiesto da un cliente e gestito da un venditore.
- **before_insert_intervento**: verifica che l'intervento sia assegnato a un meccanico.
- **before_insert_impiego**: impedisce l'assegnazione in sede di utenti non dipendenti.

Controllo dello stato dell'auto:

- **before_insert_vendita**: permette la vendita solo se l'auto è disponibile.
- **after_insert_vendita**: aggiorna automaticamente lo stato dell'auto a venduta.

In caso di violazione, il trigger blocca l'operazione tramite **SIGNAL SQLSTATE '45000'**.

Implementazione del sistema informativo

AutoGarage è stato implementato come applicazione web utilizzando il framework Django, con database MySQL e interfacce realizzate tramite template HTML e Bootstrap.

L'applicazione traduce il modello logico progettato in un sistema funzionante, permettendo agli utenti di accedere a funzionalità diverse in base al proprio ruolo.

Tecnologie utilizzate:

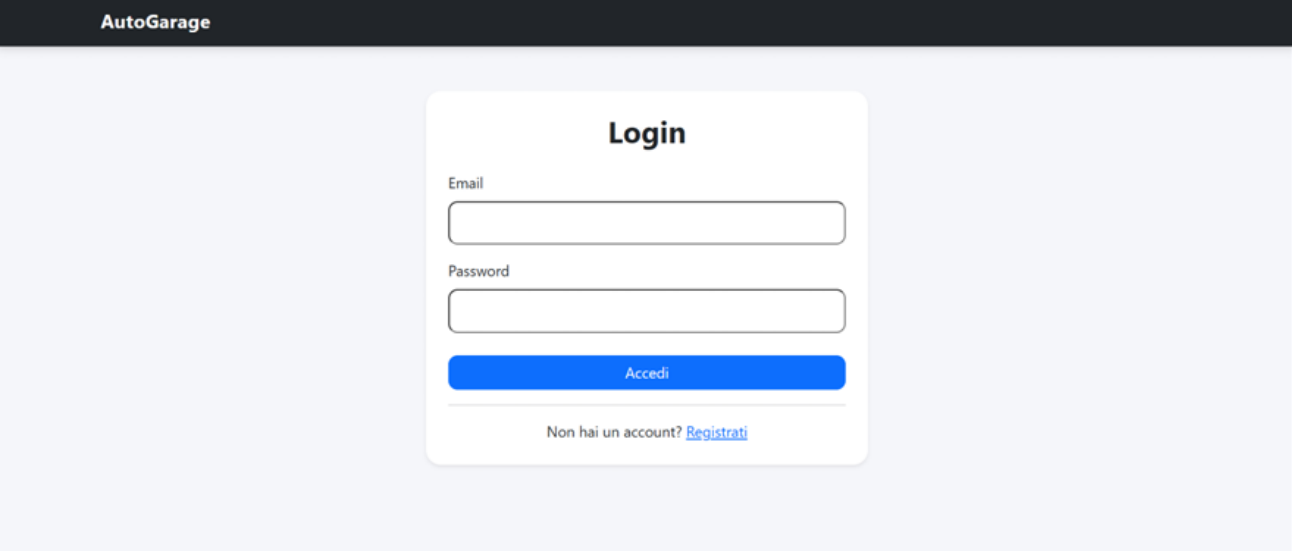
- **Python** per la logica applicativa e lo sviluppo backend
- **Django** per la gestione backend e l'interazione con il database
- **MySQL** per la base di dati relazionale
- **HTML / CSS / Bootstrap** per l'interfaccia grafica

Login e autenticazione utenti

Il sistema consente agli utenti registrati di accedere tramite email e password.

Dopo l'autenticazione, l'utente viene indirizzato alle funzionalità previste dal proprio ruolo (cliente, venditore o meccanico).

Sono inoltre presenti controlli di sicurezza per limitare tentativi di accesso non autorizzati.



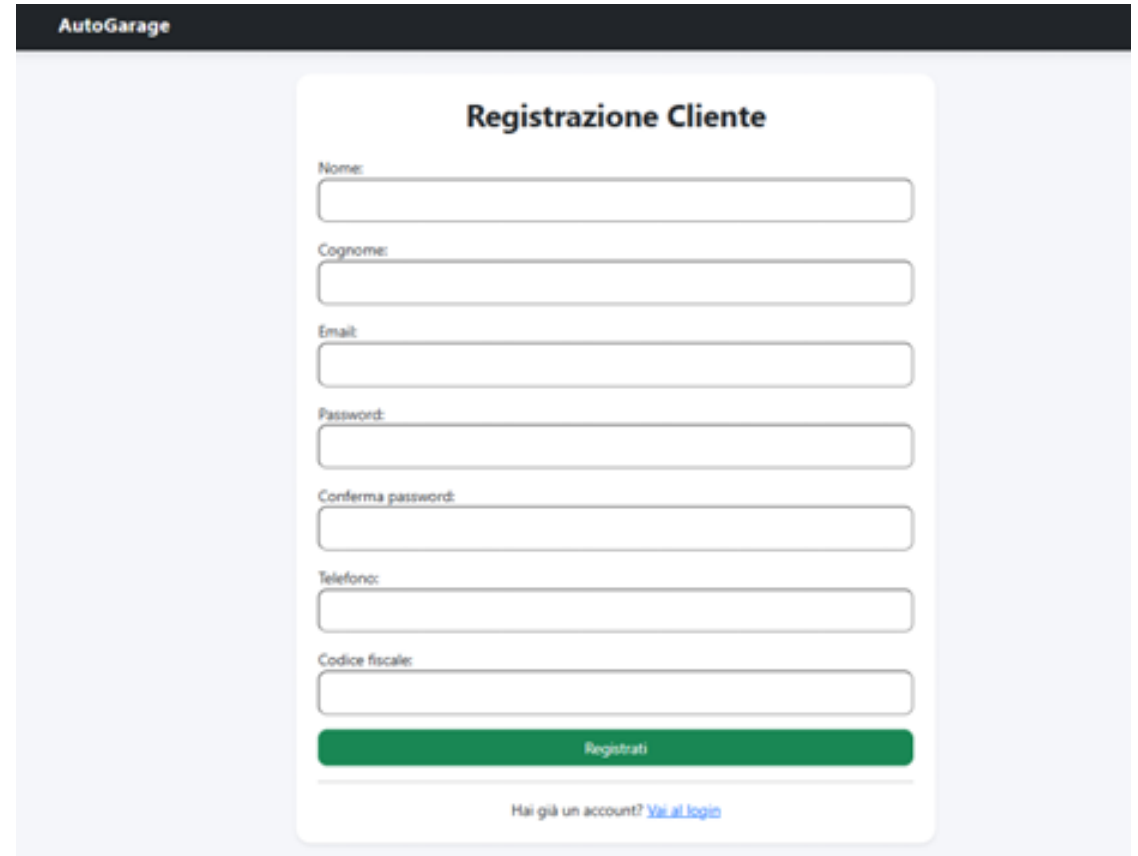
The screenshot shows a web application interface for 'AutoGarage'. At the top, there is a dark header with the text 'AutoGarage' in white. Below the header, the main content area has a light blue background. In the center, there is a white rectangular box with rounded corners containing the login form. The form is titled 'Login' in bold black text. It features two input fields: 'Email' and 'Password', each with a placeholder text and a light blue border. Below the password field is a blue button with the text 'Accedi' in white. At the bottom of the form, there is a link that says 'Non hai un account? [Registrati](#)'.

Registrazione utenti

Il sistema consente la registrazione di nuovi clienti mediante la compilazione di un'apposita form contenente i dati anagrafici e di accesso.

Durante la registrazione vengono effettuati controlli di validazione sui dati inseriti, verificando la correttezza delle informazioni e l'unicità dell'indirizzo email.

Le password vengono memorizzate in forma cifrata tramite hashing.



The screenshot displays a web interface for user registration. At the top, a dark header bar contains the text "AutoGarage". Below this, a white card titled "Registrazione Cliente" is centered. The card contains a series of input fields for personal and contact information: "Nome:", "Cognome:", "Email:", "Password:", "Conferma password:", "Telefono:", and "Codice fiscale:". Each label is followed by a corresponding text input box. Below the "Codice fiscale" field is a prominent green button labeled "Registrati". At the bottom of the card, there is a link that reads "Hai già un account? [Vai al login](#)".

Catalogo auto

Il sistema mette a disposizione un catalogo contenente tutte le automobili presenti nelle diverse sedi della concessionaria.

Per ciascun veicolo vengono visualizzate le principali informazioni, consentendo agli utenti di consultare rapidamente le automobili disponibili.

The screenshot displays the 'AutoGarage' web application interface. At the top, a dark header bar contains the 'AutoGarage' logo on the left and a user greeting 'Benvenuto Matteo' with a 'Logout' button on the right. Below the header, a white section titled 'Catalogo AutoGarage' includes a subtitle: 'Esplora le auto presenti nelle nostre sedi e visualizza le informazioni principali di ogni veicolo.' A search bar labeled 'Cerca per marca' is present, with a placeholder text 'Esempio: Audi, BMW, Alfa Romeo' and two buttons: 'Cerca' (blue) and 'Mostra tutto' (grey). The main content area features a grid of six car listings, organized into two rows of three. Each listing card has a dark header with the brand name (Alfa Romeo or Audi) and displays the following details: car model, year ('Anno'), mileage ('Chilometraggio'), price ('Prezzo'), status ('Stato'), and dealership ('Sede'). Each card also includes a yellow 'Usata' button and a blue 'Dettagli' button. The first row lists Alfa Romeo models: Giulia (2021, 59000 km, €31900.00, Caserta), Stelvio (2020, 76000 km, €34900.00, Salerno), and Tonale (2024, 0 km, €38900.00, Napoli). The second row lists Audi models: A1 (2024, 0 km, €27900.00, Caserta), A3 (2020, 55000 km, €24900.00, Napoli), and Q2 (2024, 0 km, €33500.00, Napoli).

Alfa Romeo	Alfa Romeo	Alfa Romeo
Alfa Romeo Giulia Anno: 2021 Chilometraggio: 59000 km Prezzo: € 31900.00 Stato: Disponibile Sede: AutoGarage Caserta Usata Dettagli	Alfa Romeo Stelvio Anno: 2020 Chilometraggio: 76000 km Prezzo: € 34900.00 Stato: Disponibile Sede: AutoGarage Salerno Usata Dettagli	Alfa Romeo Tonale Anno: 2024 Chilometraggio: 0 km Prezzo: € 38900.00 Stato: Disponibile Sede: AutoGarage Napoli Nuova Dettagli

Audi	Audi	Audi
Audi A1 Anno: 2024 Chilometraggio: 0 km Prezzo: € 27900.00 Stato: Noleggiata Sede: AutoGarage Caserta	Audi A3 Anno: 2020 Chilometraggio: 55000 km Prezzo: € 24900.00 Stato: Disponibile Sede: AutoGarage Napoli	Audi Q2 Anno: 2024 Chilometraggio: 0 km Prezzo: € 33500.00 Stato: Noleggiata Sede: AutoGarage Napoli

Scheda dettagliata veicolo

La scheda dettagliata consente di consultare tutte le informazioni relative a una specifica automobile presente nel catalogo.

L'utente può visualizzare le caratteristiche tecniche del veicolo e verificare la disponibilità dell'auto prima di procedere con eventuali operazioni.

AutoGarageBenvenuto Matteo [Logout](#)

Alfa Romeo Giulia

Anno: 2021

Chilometraggio: 59000 km

Alimentazione: diesel

Cambio: automatico

Colore: rosso

Prezzo: € 31900.00

Tipo: Usata

Stato: Disponibile

Sede: AutoGarage Caserta

[Torna al catalogo](#) [Prenota test drive](#)

Recensioni

Nessuna recensione presente per questa auto.

Prenotazione Test Drive

Il sistema permette ai clienti registrati di prenotare un test drive per le automobili disponibili nel catalogo.

La prenotazione viene registrata nel database e associata al cliente, al veicolo selezionato e al venditore incaricato della gestione della prova.

AutoGarageBenvenuto Matteo Logout

Prenota test drive

Scegli una data per provare il veicolo selezionato.

Alfa Romeo Giulia
Anno: 2021
Prezzo: € 31900.00
Sede: AutoGarage Caserta

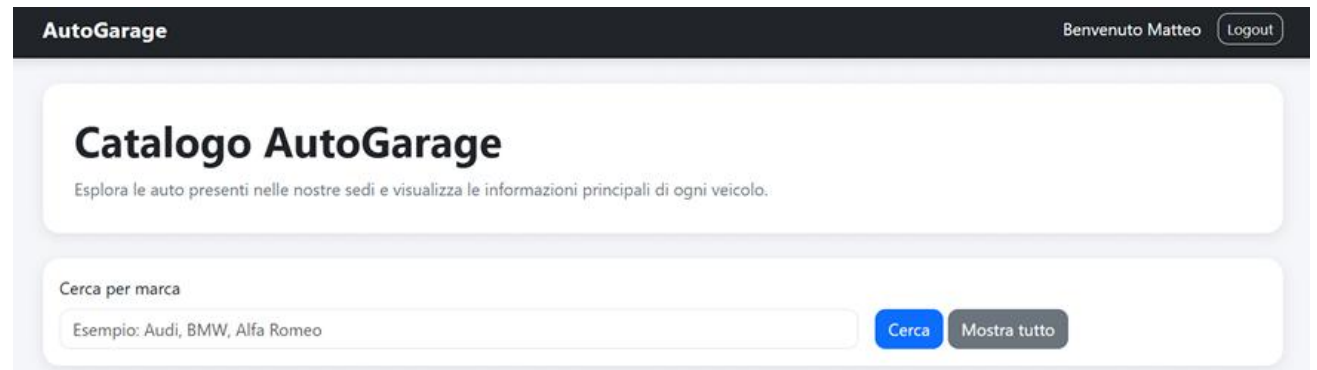
Data test drive:

Conferma prenotazioneAnnulla

Ricerca auto per marca

Il sistema consente agli utenti di filtrare il catalogo delle automobili in base alla marca del veicolo.

La funzionalità permette una consultazione più rapida del catalogo e facilita l'individuazione delle auto di interesse.



The screenshot displays the 'AutoGarage' website interface. At the top, a dark header bar contains the 'AutoGarage' logo on the left and a user login status 'Benvenuto Matteo' with a 'Logout' button on the right. Below the header, a large white box features the title 'Catalogo AutoGarage' and a subtitle 'Esplora le auto presenti nelle nostre sedi e visualizza le informazioni principali di ogni veicolo.' Underneath this, a search section titled 'Cerca per marca' includes a text input field with the placeholder text 'Esempio: Audi, BMW, Alfa Romeo'. To the right of the input field are two buttons: a blue 'Cerca' button and a grey 'Mostra tutto' button.

Visualizzazione recensioni

Il sistema consente agli utenti di visualizzare le recensioni associate alle automobili presenti nel catalogo.

Ogni recensione contiene una valutazione numerica e un commento testuale, utili per fornire informazioni aggiuntive sull'esperienza degli utenti con il veicolo.

Recensioni

Voto: 4/5

Test drive organizzato bene, personale gentile.

Scritta da Alessia Bruno

Dashboard venditore

La dashboard venditore fornisce una vista centralizzata delle principali attività commerciali della concessionaria.

Attraverso questa sezione è possibile consultare e gestire le operazioni di vendita, noleggio e prenotazione dei test drive associate ai clienti e ai veicoli presenti nel sistema.

AutoGarage

Benvenuto Marco [Logout](#)

Dashboard Venditore

Riepilogo dei test drive, delle vendite e dei noleggi gestiti.

Test drive gestiti

Alfa Romeo Tonale

Cliente: Francesco Greco

Data test drive: Dec. 10, 2025

Vendite gestite

Volkswagen T-Roc

Cliente: Francesco Greco

Data vendita: Oct. 2, 2025

Prezzo: € 31500.00

Fiat Panda

Cliente: Francesco Greco

Data vendita: Sept. 10, 2025

Prezzo: € 15500.00

Noleggi gestiti

BMW X1

Cliente: Sara Lombardi

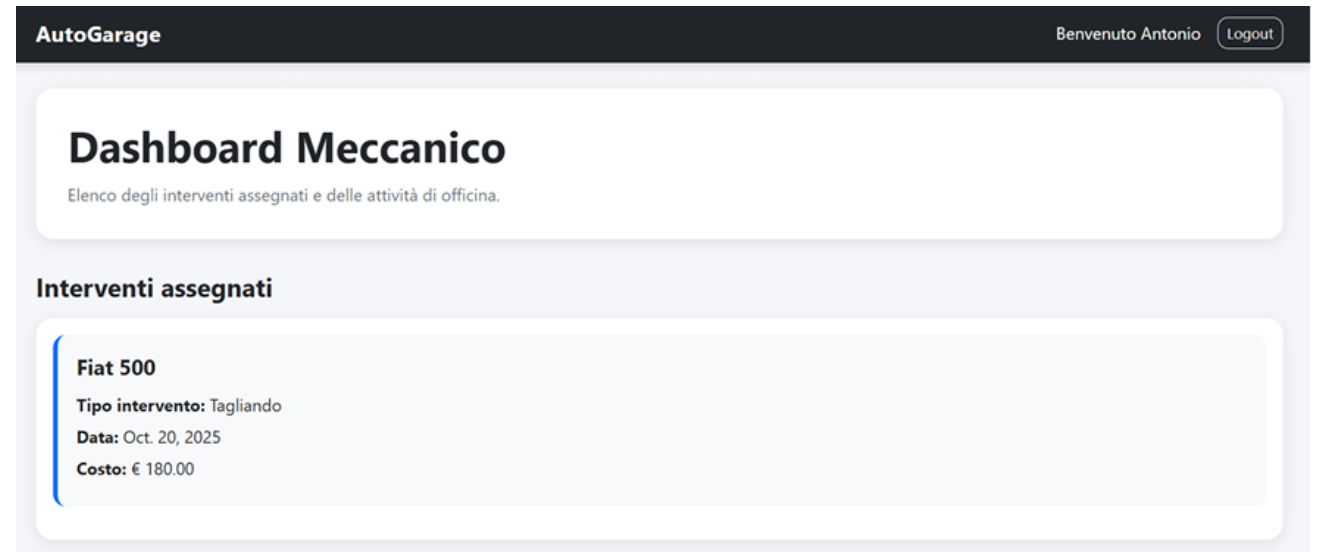
Periodo: Nov. 7, 2025 - Nov. 10, 2025

Costo totale: € 320.00

Dashboard meccanico

La dashboard meccanico consente la gestione e la consultazione degli interventi di manutenzione e riparazione assegnati.

Il personale tecnico può visualizzare le informazioni relative ai veicoli e alle attività da svolgere.



Sicurezza del sistema

Per garantire l'affidabilità dell'applicazione sono state adottate diverse misure di sicurezza volte a proteggere il sistema da attacchi comuni alle applicazioni web.

Attacchi considerati:

- **SQL Injection**
- **Attacchi di forza bruta**
- **Attacchi a dizionario**

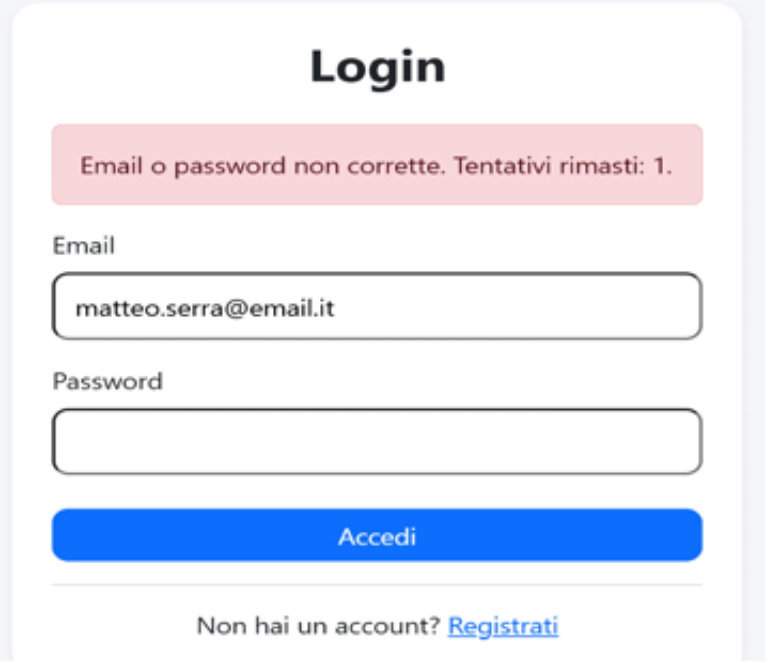
Contromisure adottate:

- Utilizzo dell'ORM di Django
- Limitazione dei tentativi di accesso
- Memorizzazione sicura delle password tramite hashing

SQL Injection (1/2)

L'attacco **SQL Injection** consiste nell'inserimento di istruzioni SQL malevole all'interno dei campi di input con l'obiettivo di modificare il comportamento delle query eseguite dal database.

Nel sistema AutoGarage questo rischio è mitigato mediante l'utilizzo dell'**ORM di Django**, che genera automaticamente query parametrizzate e separa i dati inseriti dall'utente dalla struttura delle interrogazioni SQL.



The image shows a web login interface. At the top, the word "Login" is centered in a bold, black font. Below it, a light red rectangular box contains the error message "Email o password non corrette. Tentativi rimasti: 1." in a small, black font. Underneath the error box, there are two input fields. The first is labeled "Email" and contains the text "matteo.serra@email.it". The second is labeled "Password" and is currently empty. Below the password field is a blue rectangular button with the white text "Accedi". At the bottom of the form, there is a link that says "Non hai un account? [Registrati](#)".

SQL Injection (2/2)

Il sistema AutoGarage non costruisce query SQL manualmente a partire dagli input dell'utente.

I dati inseriti nel form di login vengono validati tramite Django Forms e successivamente verificati attraverso la funzione `authenticate()`, riducendo il rischio che un input malevolo venga interpretato come codice SQL.

```
email = form.cleaned_data["email"]  
password = form.cleaned_data["password"]
```

I dati inseriti dall'utente vengono controllati e sanitizzati prima di essere utilizzati dall'applicazione

```
utente = authenticate(request, username=email, password=password)  
  
if utente is None:  
    raise Utente.DoesNotExist
```

Le credenziali vengono verificate tramite la funzione `authenticate()`, senza costruire manualmente query SQL

Attacco a forza bruta (1/2)

L'attacco di forza bruta consiste nel tentare ripetutamente diverse combinazioni di credenziali fino a individuare quelle corrette.

Per verificare la resistenza del sistema sono stati effettuati diversi tentativi consecutivi di accesso con password errate.

Risultato:

- Tentativi di login multipli con credenziali non valide
- Incremento del contatore dei tentativi falliti
- Nessun accesso non autorizzato ottenuto

The image displays three sequential screenshots of a web application's login page, illustrating the progression of a brute force attack. Each screenshot shows a login form with fields for 'Email' and 'Password', an 'Accedi' button, and a link to 'Registrati'.

- Top Left Screenshot:** The message above the form reads: "Email o password non corrette. Tentativi rimasti: 2." The email field contains "marbia@email.it".
- Top Right Screenshot:** The message above the form reads: "Email o password non corrette. Tentativi rimasti: 1." The email field contains "marbia@email.it".
- Bottom Screenshot:** The message above the form reads: "Hai effettuato 3 tentativi errati. Account bloccato per 2 minuti." The email field contains "marbia@email.it".

Attacco a forza bruta (2/2)

Per limitare gli attacchi di forza bruta il sistema monitora il numero di tentativi falliti associati a ciascun account.

Dopo un determinato numero di errori consecutivi, l'accesso viene temporaneamente bloccato.

```
failed_key = f"failed_login_{email}"
block_key = f"blocked_login_{email}"

if cache.get(block_key):
    errore = "Troppi tentativi falliti. Riprova tra 2 minuti."
```

Verifica della presenza di un blocco temporaneo e visualizzazione del relativo messaggio di errore

```
failed_attempts = cache.get(failed_key, 0) + 1

if failed_attempts >= 3:
    cache.set(block_key, True, timeout=120)
```

Attivazione automatica del blocco dopo tre tentativi consecutivi falliti per una durata di 120 secondi

Attacco a dizionario (1/2)

L'attacco a dizionario consiste nel tentare l'accesso utilizzando elenchi di password comuni e frequentemente utilizzate dagli utenti.

Per ridurre il rischio di successo di questo attacco, AutoGarage impone specifici requisiti di complessità durante la registrazione delle password e memorizza le credenziali in forma hashata all'interno del database.

```
if len(password) < 8:
    raise forms.ValidationError("La password deve contenere almeno 8 caratteri.")

if nome and nome in password_lower:
    raise forms.ValidationError("La password non può contenere il nome dell'utente.")

if cognome and cognome in password_lower:
    raise forms.ValidationError("La password non può contenere il cognome dell'utente.")

if parte_email and parte_email in password_lower:
    raise forms.ValidationError("La password non può contenere la parte iniziale dell'email.")

if not re.search(r"[A-Z]", password):
    raise forms.ValidationError("La password deve contenere almeno una lettera maiuscola.")

if not re.search(r"[a-z]", password):
    raise forms.ValidationError("La password deve contenere almeno una lettera minuscola.")

if not re.search(r"\d", password):
    raise forms.ValidationError("La password deve contenere almeno un numero.")

if not re.search(r"[^\w\s]", password):
    raise forms.ValidationError("La password deve contenere almeno un carattere speciale.")
```

*Verifica dei requisiti minimi di sicurezza
della password durante la registrazione*

Attacco a dizionario (2/2)

Oltre ai controlli di complessità, le password non vengono memorizzate in chiaro nel database.

Prima del salvataggio viene applicata la funzione di hashing fornita da Django, che trasforma la password in una rappresentazione non reversibile.

In questo modo, anche in caso di accesso non autorizzato al database, le credenziali originali non risultano direttamente leggibili.

```
utente.password = make_password(self.cleaned_data["password"])
```

*Memorizzazione delle password tramite
hashing prima del salvataggio nel database*